

Security Audit

Urbit Token (NFT Fractionalization)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	14
Conclusion	15
Our Methodology	16
Disclaimers	18
About Hashlock	19

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Urbit Token team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Urbit Token is a tokenization system that converts Urbit stars (ERC721 points in the Azimuth registry) into fungible ERC20 tokens. Users deposit a star into the Vault contract to receive 65,535 URBIT tokens (representing the star's spawnable planets) plus 1 USTAR token (representing the star itself). Burning the same amounts redeems any star held by the vault. There is no pre-mint — tokens are minted on deposit and burned on redemption, so total supply always reflects the number of stars held.

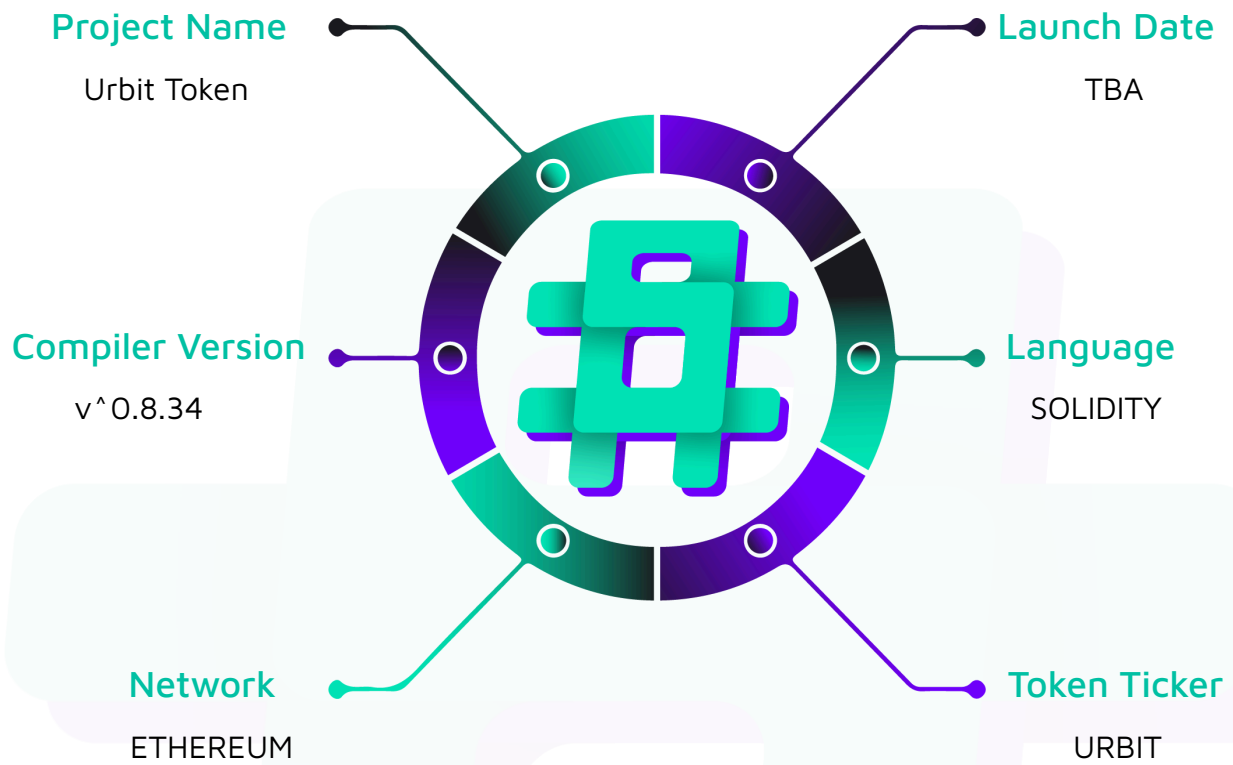
Project Name: Urbit Token

Project Type: NFT Fractionalization

Compiler Version: ^0.8.34

Logo:



Visualised Context:

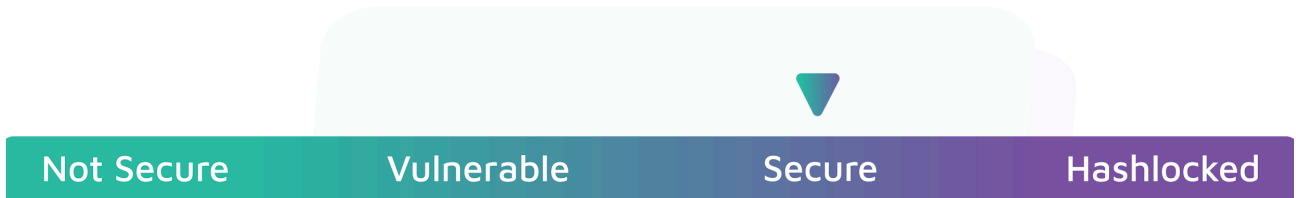
Audit Scope

We at Hashlock audited the solidity code within the Urbit Token project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Urbit Token Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	March, 2026
Contract 1	UrbitToken.sol
Contract 2	UstarToken.sol
Contract 3	UrbitVault.sol
Audited GitHub Commit Hash	ac52dd5af6075a759e9c4a2b05dd7ec9b4fc9f70
Fix Review GitHub Commit Hash	11c52c616f1b7566810f39ac99c27403d0160932

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
UrbitVault.sol - Allows the owner to: - Add or remove addresses from the deposit whitelist - Toggle the deposit whitelist on or off - Renounce ownership - Transfer ownership via Two-Step - Allows users to: - Deposit a virgin star and receive two tokens - Deposit multiple virgin stars in a single tx - Redeem a deposited virgin star - Redeem multiple deposited stars in a single tx - Check if a star is eligible for a deposit	Contract achieves this functionality.
UrbitToken.sol - Typical ERC20 Contract with Permit functionalities - Allows OWNER (Vault) to MINT and BURN tokens	Contract achieves this functionality.
UstarToken.sol - Typical ERC20 Contract with Permit functionalities - Allows OWNER (Vault) to MINT and BURN tokens	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Urbit Token project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Urbit Token project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

QA

[Q-01] UrbitVault - Redundant ERC20 Inheritance in Token Contracts

Description

Both `UrbitToken` and `UstarToken` explicitly inherit `ERC20` alongside `ERC20Permit` and `Ownable` (e.g., `contract UrbitToken is ERC20, ERC20Permit, Ownable`). However, OpenZeppelin's `ERC20Permit` already inherits `ERC20` in its own declaration (`abstract contract ERC20Permit is ERC20, IERC5267, Nonces`). Listing `ERC20` in the `is` clause is therefore redundant — `ERC20Permit` already brings the full `ERC20` implementation into the contract's linearization. While Solidity's C3 linearization deduplicates the base correctly and no functional bug arises, the explicit `ERC20` inclusion adds unnecessary noise to the inheritance graph and may mislead readers into thinking `ERC20` is a separate, independent dependency from `ERC20Permit`.

Impact

This is purely cosmetic and has zero effect on compiled bytecode, gas cost, or runtime behavior. Solidity's C3 linearization algorithm handles the duplicate base contract correctly, so no double-initialization or storage collision occurs. The only consequence is reduced code clarity — a developer reading the contract may not immediately recognize that `ERC20` is already provided transitively through `ERC20Permit`, leading to a slightly misleading mental model of the inheritance hierarchy.

Recommendation

Remove the explicit `ERC20` from the inheritance list in both token contracts, changing the declarations to `contract UrbitToken is ERC20Permit, Ownable` and `contract UstarToken is ERC20Permit, Ownable`. The `ERC20(...)` call in the constructor initializer must remain, as it sets the token name and symbol for the `ERC20` base that `ERC20Permit` inherits. This change simplifies the inheritance chain without altering any behavior.

Status

Resolved



Centralisation

The Urbit Token project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Urbit Token project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.